



Household Electricity Usage Calculator and Energy Saving Recommendation System

Sustainable Development Goal:
SDG 7 – Affordable and Clean Energy

MEMBERS:

Castro Vedal Jr B.
Sentasas, Johndyl Glance C.
Tiu, Charles Jayzee Lei M.
Serrano, Samuel Johnley B.
Aceveda, Jayrol H.

Section:
1.3 BSIT

project overview:

This project focuses on developing a user-friendly tracking tool that helps households identify their most power-hungry appliances and calculates daily electricity consumption, while automatically generating personalized, habit-changing recommendations based on the user's specific savings goals to significantly lower their monthly energy expenses.

problem statement:

Many households are unaware of how much electricity their appliances consume daily, which can lead to high electricity bills and difficulty in achieving savings goals. Without proper monitoring, users cannot easily identify which appliances contribute the most to energy consumption. This project aims to solve this problem by providing a system that calculates electricity usage and gives recommendations based on the user's desired savings. The system helps users make better decisions in reducing energy consumption and managing their electricity expenses.

Main function:**The Main Logic (Household Electricity Usage.cpp)**

Think of the `main()` function as the boss or the skeleton of the whole program. It controls the order of everything that happens from the moment you hit run.

#include "Main.h" and "Database.h": We added these custom headers so this file can see and use the functions written in the other files. Without these, `main()` wouldn't know what `registerUser()` or `inputData()` means.

switch (startChoice): We used this for the main entrance. It checks if you want to Register (1) or Login (2) right away. If login fails, return 0; kicks you out of the program immediately.

if (startChoice == 1) (Login after Register): We put this buffer here because if a new user registers, they aren't automatically logged in. This forces them to log in with their newly created credentials before moving forward.

Arrays (appliances[MAX], watts[MAX], hours[MAX]): We created parallel arrays here to temporarily hold the data in the computer's memory while the program is running. For example, `appliances[0]`, `watts[0]`, and `hours[0]` all belong to the exact same device.

do-while Loop: We used this loop for the main menu because we want the user to keep choosing options (like inputting data, computing, checking reports) over and over again until they specifically choose option 5 to save or exit.

The Core Functions (Function.cpp)

This file handles all the math and the heavy lifting for managing appliance data.

#include <fstream> and <iomanip>: We added fstream so we can write the data to a text file later, and iomanip so we can use fixed and setprecision(2) to make the PHP money layout look clean with exactly two decimal places.

inputData(): This fills up our arrays. We added while loops inside to validate inputs because users can be clumsy it stops them from entering impossible values like negative wattage or negative hours. cin.ignore() is used to clear out the "Enter" key space so getline() doesn't accidentally skip the appliance name.

computeUsage(): This converts total watts to kilowatts (watts / 1000.0) and multiplies it by the hours used. It loops through all entered appliances and aggregates them into totalUsage.

displayReport(): This handles the logic for the energy budget. It calculates the cost based on a fixed rate of 12.0 PHP per kWh. We threw an if-else statement here to instantly give advice if the user spent more than their target budget.

saveUserData(): This is where ios::app (append mode) comes in. We put this so that every time a user saves, C++ appends (adds) the data to the bottom of users.txt instead of wiping out everything previously saved.

Authentication and Database (Database.cpp)

This file handles the security stuff—making sure users exist and passwords match.

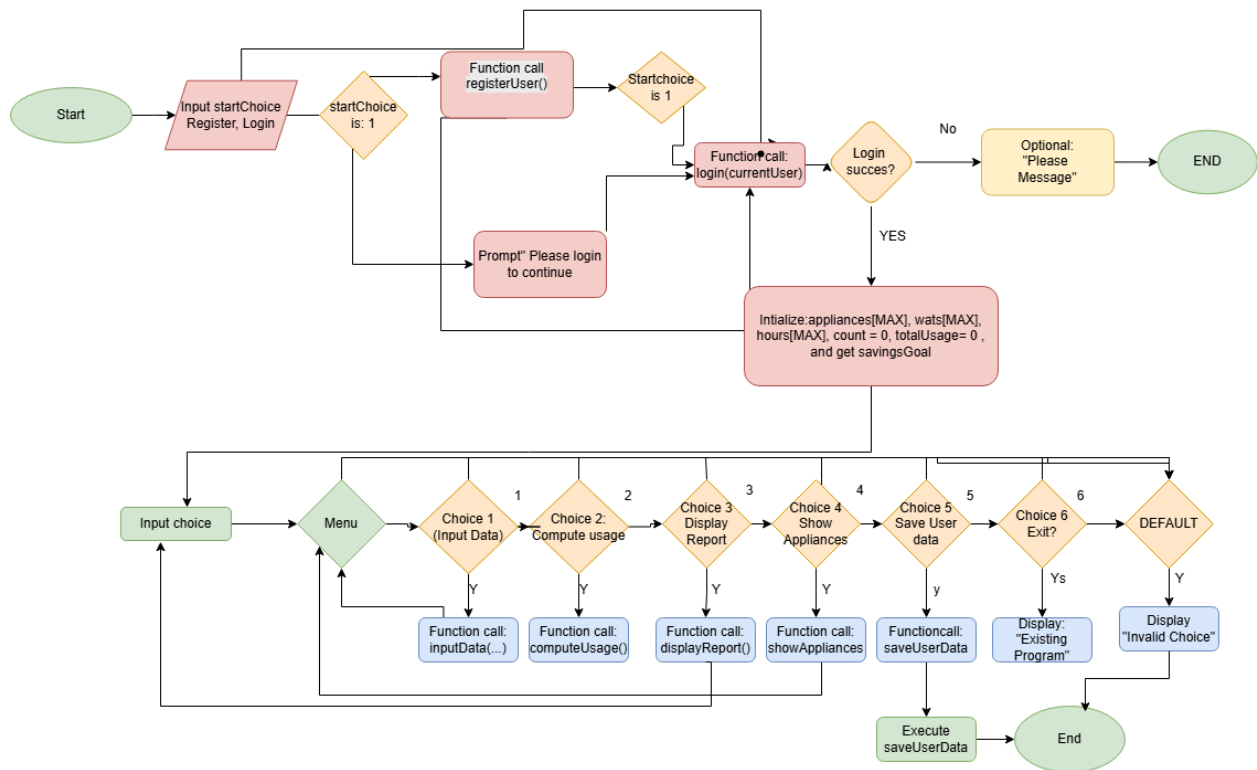
registerUser(): This opens users.txt using ifstream to check if a username is already taken. If it's free, it switches to ofstream and saves the new credentials as a simple text pair (username password).

login(): We added an attempts = 3 counter here to make it secure. The while (readFile >> fileUser >> filePass) loop reads through the text file line by line to see if the typed username and password match any stored pairs. If you fail 3 times, it locks you out.

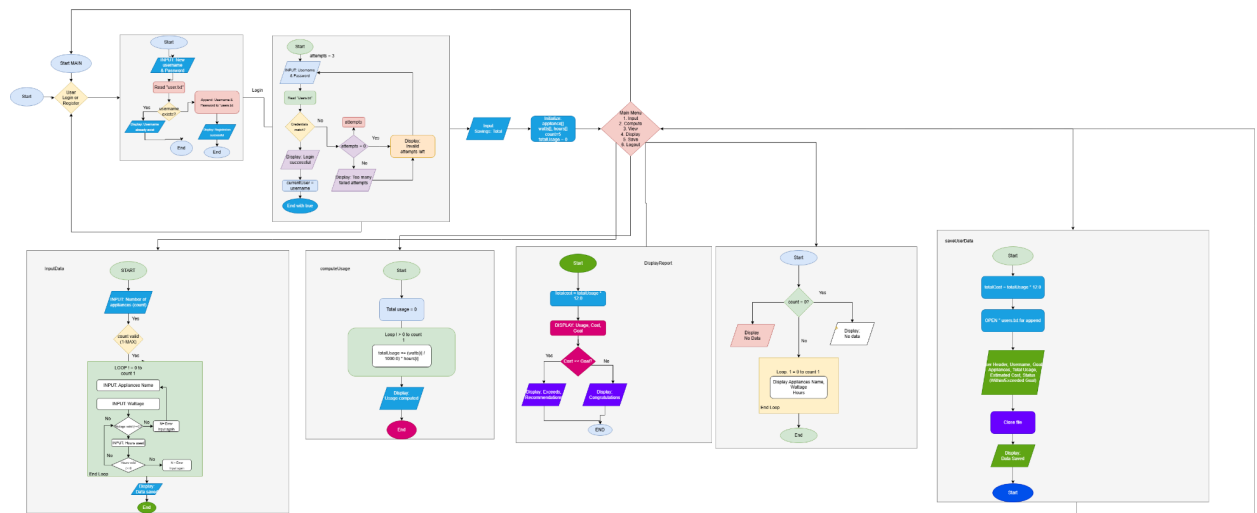
The Header Files (Database.h and Main.h)

#ifndef and #define: These are called include guards. We wrapped both headers in these so that even if multiple files call Main.h or Database.h, C++ only reads them once. It prevents duplicate definition errors during compilation.

Function Prototypes: Think of these files as the menu of a restaurant. They don't cook the food (code the logic); they just list the names of the functions so other parts of the project know they exist and can be called anywhere.



FLOWCHART



System flow

Start the application.

Prompt user to either Register or Login.

If registration is chosen, call the registerUser() function. Once registered, proceed to login.

Call the login(currentUser) function. The user has up to 3 attempts to log in successfully.

If login fails after maximum attempts, display an error message and End the program.

If login is successful, initialize global arrays and variables: appliances[MAX], wats[MAX], hours[MAX], count = 0, totalUsage = 0, and prompt the user for their savingsGoal.

Enter the Main Menu Loop and display the following choices:

- 1: Input Data (Calls inputData())
- 2: Compute Usage (Calls computeUsage())
- 3: Display Report (Calls displayReport())
- 4: Show Appliances (Calls showAppliances())
- 5: Save User Data (Calls saveUserData())
- 6: Exit Program

Loop until the user chooses option 6 (Exit) or choice 5 completes execution.

```

START
CALL loginAndRegistration()
IF loginSuccessful == TRUE THEN
    // Initialization
    DECLARE appliances[MAX] AS STRING
    DECLARE wats[MAX] AS REAL
    DECLARE hours[MAX] AS REAL
    DECLARE count AS INTEGER = 0
    DECLARE totalUsage AS REAL = 0.0

    INPUT savingsGoal

    DECLARE choice AS INTEGER = 0
    WHILE choice != 6 DO
        PRINT "Main Menu: 1. Input, 2. Compute, 3. View, 4. Display, 5. Save, 6. Logout"
        INPUT choice

        SWITCH choice DO
            CASE 1:
                CALL inputData(appliances, wats, hours, count)
            CASE 2:
                totalUsage = CALL computeUsage(wats, hours, count)
            CASE 3:
                CALL displayReport(totalUsage, savingsGoal)
            CASE 4:
                CALL showAppliances(appliances, wats, hours, count)
            CASE 5:
                CALL saveUserData(appliances, wats, totalUsage, savingsGoal)
                Exit Loop // As indicated by the execute/end flow in image 2
            CASE 6:
                PRINT "Exiting Program"
            DEFAULT:
                PRINT "Invalid Choice"
        END SWITCH
    END WHILE
ELSE
    PRINT "Access Denied. Program Terminated."
END IF
END

```

2. Register Login Algorithm

Registration: Prompt for a new username and password, check if the username already exists, and save credentials.

Login: Allow the user to input credentials. Compare against saved data. Track attempts (max 3). Return success status

Pseudocode

```
FUNCTION registerUser()
  INPUT newUsername, newPassword
  IF usernameExists(newUsername) THEN
    PRINT "Username already exists!"
  ELSE
    SaveCredentials(newUsername, newPassword)
    PRINT "Registration Successful"
  END IF
END FUNCTION

FUNCTION login() RETURNS BOOLEAN
  DECLARE attempts AS INTEGER = 3
  DECLARE loginSuccessful AS BOOLEAN = FALSE

  WHILE attempts > 0 AND loginSuccessful == FALSE DO
    INPUT username, password
    IF CredentialsMatch(username, password) THEN
      PRINT "Login Successful"
      loginSuccessful = TRUE
    ELSE
      attempts = attempts - 1
      PRINT "Invalid credentials. Remaining attempts: ", attempts
    END IF
  END WHILE

  IF loginSuccessful == FALSE THEN
    PRINT "Too many failed attempts."
```

END IF

RETURN loginSuccessful

END FUNCTION

Module 1: Input Data

Algorithm

Prompt the user for the number of appliances to input (num).

Validate that num is within legal limits (1 to MAX).

Loop from 1 to num:

Input the appliance name.

Input the wattage. Validate that wattage is positive.

Input the operational hours. Validate that hours are positive ($0 \leq \text{hours} \leq 24$).

Update the global array elements and the global tracker count.

Pseudocode

FUNCTION inputData(REF appliances, REF wats, REF hours, REF count)

INPUT num

IF num < 1 OR num > MAX THEN

PRINT "Count invalid!"

ELSE

FOR i = 1 TO num DO

INPUT applianceName

appliances[count] = applianceName

// Input Wattage validation loop

REPEAT

INPUT applianceWattage

IF applianceWattage < 0 THEN

PRINT "Error: Invalid Wattage"

END IF

UNTIL applianceWattage >= 0

wats[count] = applianceWattage

// Input Hours validation loop

REPEAT

INPUT applianceHours

IF applianceHours < 0 OR applianceHours > 24 THEN

PRINT "Error: Invalid Hours"

END IF

UNTIL applianceHours >= 0 AND applianceHours <= 24

hours[count] = applianceHours

count = count + 1

```

        END FOR
        PRINT "Data saved successfully"
    END IF
END FUNCTION

```

Module 2 & 3: Compute Usage & Display Report

Compute: Loop through all recorded items up to count. Accumulate total energy consumption formula: $\text{Total Usage} = \sum (\text{wattage} \times \text{hours}) / 1000$.

Report: Compare computed total consumption against user savings goal. If total consumption is less than or equal to the savings goal, display a "Congratulations" message. Otherwise, display recommendations to decrease energy usage.

PSEUDOCODE

```

FUNCTION computeUsage(wats, hours, count) RETURNS REAL
    DECLARE totalUsage AS REAL = 0.0
    FOR i = 0 TO count - 1 DO
        totalUsage = totalUsage + ((wats[i] * hours[i]) / 1000.0)
    END FOR
    PRINT "Usage computed: ", totalUsage
    RETURN totalUsage
END FUNCTION

```

```

FUNCTION displayReport(totalUsage, savingsGoal)
    PRINT "Total Usage: ", totalUsage, " Goal: ", savingsGoal
    IF totalUsage <= savingsGoal THEN
        PRINT "Congratulations! You met your savings goal."
    ELSE
        PRINT "Conserve: Recommendations on lowering your usage."
    END IF
END FUNCTION

```

Module 4 & 5: Show Appliances & Save User Data

Show Appliances: Check if count is 0. If it is, output "No Data". Otherwise, use a loop to print out names, wattages, and run-hours for all stored appliances.

Save User Data: Open/create a structured storage file. Write a standard data header containing username, followed by lists of appliances, total usage metrics, and goals. Safely close the storage link.

PSEUDOCODE

```
FUNCTION showAppliances(appliances, wats, hours, count)
  IF count == 0 THEN
    PRINT "No Data"
  ELSE
    FOR i = 0 TO count - 1 DO
      PRINT "Appliance Name: ", appliances[i]
      PRINT "Wattage: ", wats[i]
      PRINT "Hours Used: ", hours[i]
    END FOR
  END IF
END FUNCTION
```

```
FUNCTION saveUserData(appliances, wats, totalUsage, savingsGoal)
  // Assume global or active user session variable 'username'
  OPEN "user_list.txt" FOR APPEND AS file

  WRITE file, "--- Header ---"
  WRITE file, "Username: ", username
  WRITE file, "Appliances List: ", appliances
  WRITE file, "Total Usage: ", totalUsage, " kWh"
  WRITE file, "Savings Goal Status: ", savingsGoal

  CLOSE file
  PRINT "Data Saved to File"
END FUNCTION
```